

Vinnitsia National Technical University
Faculty of Information Technology and Computer Engineering
Department of Software Engineering

COURSE PROJECT
in the course “**Object-Oriented** Programming”
on the topic: “DEVELOPMENT OF A SOFTWARE SYSTEM FOR MODELING
OF OBJECTS: ‘TORNADO GYM Simulator. Macro-objects: Front Desk & Intake, Cardio
Area, Weights Area. Micro-objects: Athlete, Power Lifter, Gym Boss’
USING UML AND THE JAVA PROGRAMMING LANGUAGE”

Completed by: 1st-year student, Group 2PI-25b
Major: F2 Software Engineering
Yaroslav SYCH



Advisor: Associate Professor of the Department of
Software Engineering
Ph.D. in Engineering, Associate Professor Oleksandr
RESHETNIK

Score: _82_
Grade: ECTS __B__

Committee Members:

(signature)

O.O. Reshetnik

(signature)

D.I. Katelnikov

(signature)

(last name and initials)

Vinnytsia – 2026

Vinnitsia National Technical University
Faculty of Information Technology and Computer Engineering
Department of Software Engineering
First Level of Higher Education (Bachelor's Degree)
Major F2: Software Engineering
Educational and Professional Program "Software Engineering"

APPROVED

Head of the Department of Software Engineering
O.N. Romanyuk, Ph.D. in Technical Sciences, Professor

February "10," 2026

INDIVIDUAL ASSIGNMENT

for a term paper in the course "Object-Oriented Programming"
To Yaroslav Oleksandrovysh Sychu, student of class 2PI-25b

Thesis Topic: DEVELOPMENT OF A SOFTWARE SYSTEM FOR OBJECT MODELING: "TORNADO GYM Simulator." Macro-objects: Front Desk & Intake, Cardio Area, Weights Area. Micro-objects: Athlete, Power Lifter, Gym Boss" USING UML AND THE JAVA PROGRAMMING LANGUAGE

Task: Develop a software system and an explanatory note on the development process.

Input data for the project:

Programming language: Java.

Development environment: NetBeans IDE, IntelliJ IDEA, Eclipse, or Visual Studio Code.

Graphical interface: JavaFX, Swing, or AWT.

Micro-object image: at least three graphical primitives (one of which must be text). The image of each macro-object must consist of at least three graphical primitives (line, rectangle, circle, arc, polygon, text, graphic image, etc.), and one of the primitives must be text. Add a reference type to the micro-object class as an element to require deep copying. Implement deep copying for the micro-object class. Macro-objects must be able to contain more than two micro-objects. A hierarchy of micro-

object classes must be implemented, containing at least three levels of inheritance that can be instantiated and displayed on the screen. The following query must be implemented: find a micro-object with specified parameters. The program searches for an object with the entered parameters and displays on the screen: its location and which macro-objects it belongs to (or indicates that the micro-object does not belong to any macro-object). The following query must be implemented: display a list of micro-objects that belong to the specified macro-object. The following query must be implemented: display a list of micro-objects that do not belong to any macro-object. The following query must be implemented: set a sorting criterion for micro-objects in queries that return lists. There must be at least three sorting criteria, for example: Student_Name, Average_Score, Number_of_Grades_of_4_or_Higher. Automatic movement of certain micro-objects must be implemented: 1) during movement, some micro-objects interact with each other; 2) some micro-objects enter macro-objects and some exit them; 3) the nature of the micro-objects' movement must change when a specific keyboard key is pressed or when a menu command is selected. You must implement at least three queries for counting micro-objects. For example: a) the number of active micro-objects; b) the number of objects with a health level above 50%; c) the number of objects located on the right side of the screen, etc. The program window displays only a specific portion of the game world, which must not exceed 25% of its total size. Implement the ability to change the portion of the game world that the user sees. A minimap must be implemented to enable fast navigation: 1) the visible area of the universal object must be marked on the minimap; 2) the movement of the visible area of the universal object updates the minimap accordingly; 3) macro-objects and micro-objects are marked on the minimap; 4) the movement and interaction of micro-objects result in corresponding changes on the minimap; 5) clicking the left mouse button on the minimap moves the visible area of the universal object. Serialization/deserialization of all objects into a text/binary/XML file must be programmed. Serialization/deserialization must preserve not only the state information of macro- and micro-objects, but also their positions on the screen.

Contents of the explanatory note:

ABSTRACT
TABLE OF CONTENTS
INTRODUCTION
1 ANALYSIS OF THE CURRENT STATE OF THE PROBLEM AND
JUSTIFICATION OF THE THESIS TOPIC
2 DEVELOPMENT OF A GRAPHICAL DISPLAY SUBSYSTEM
3 DEVELOPMENT OF CLASS, OBJECT, AND STATE DIAGRAMS
4 USER MANUAL
CONCLUSIONS
LIST OF REFERENCES
APPENDIX A (required) Program Listing

Assignment Date: February "11," 2026

Deadline for submission of the completed course project: June 18, 2026

Advisor _____ Oleksandr RESHETNIK
Signature
(signature)
Assignment received by _____ Yaroslav SYCH
Ysch
(signature)

ABSTRACT

Sych, Y.O. Development of a Software System for Object Modeling: “Tornado GYM Simulator.” Macro-objects: Front Desk & Intake, Cardio Area, Weights Area. Micro-objects: Athlete, Power Lifter, Gym Boss: Course Project in the Course “Object-Oriented Programming”

This course project involves the development of a software application for Windows/Linux/Mac OS, built using JavaFX. The application simulates the behavior of objects—“athletes, gym areas, and equipment”—using UML and the Java programming language. All the basic principles of object-oriented programming were applied, namely: abstraction, encapsulation, inheritance, and polymorphism. A downward trend in young people’s interest in a healthy lifestyle was investigated. A finished software product was created.

Keywords: micro-objects, inheritance, macro-objects, aggregation, universal object, composition.

TABLE OF CONTENTS

INTRODUCTION	5
1 ANALYSIS OF THE CURRENT STATE OF THE ISSUE AND JUSTIFICATION OF THE RESEARCH TASK	8
1.1 Subject Area	8
1.2 Development of the Technical Specifications for the Project	11
1.3 Justification for the Choice of Programming Language	15
1.4 Conclusions	19
2 DEVELOPMENT OF THE GRAPHICAL DISPLAY SUBSYSTEM	20
2.1 Graphical Display Model	20
2.2 Graphic Procedures of the Graphic Display Subsystem	26
3 DEVELOPMENT OF CLASS, OBJECT, AND STATE DIAGRAMS	28
3.1 Inheritance Diagram	28
3.2 Aggregation Diagram	30
3.3 Composition Diagram	31
3.4 Association Diagram	33
3.5 Cooperation Diagram	33
3.6 Sequence Diagram	34
3.7 State Diagram	36
4 USER MANUAL	38
4.1 Main Program Window	38
4.2 Adding a New Athlete	39
4.3 Moving Athletes	41
4.4 Editing an Athlete	41
4.5 Deleting and Cloning Athletes	43
4.6 Sorting and Statistics	43
4.7 Saving and Loading the State	44
4.8 Error Messages and Special Situations	45

CONCLUSIONS	46
LIST OF REFERENCES	47
APPENDIX A (required) Program Listing	49

INTRODUCTION

The current stage of human civilization is characterized by fundamental changes in people's lifestyles, driven by rapid advances in information technology and the automation of production processes. Global digitalization, in addition to its obvious advantages, has brought with it a number of challenges, among which perhaps the most significant is hypodynamia—a pathological condition characterized by a lack of physical activity. Most modern professions have shifted to the virtual realm, forcing the younger generation and the working-age population to spend a significant portion of their time sitting in front of screens. This creates a critical imbalance between mental exertion and physical condition.

At the same time, a shift in attitudes toward one's own body and health is taking place in the public consciousness. Physical education is no longer viewed solely as a professional activity or a means of preparing for work; it is becoming an integral part of harmonious personal development, a means of socialization, and an important hobby. However, the issue of motivation arises on the path to forming a stable habit of participating in sports. Traditional methods of engagement—lectures, calls for a healthy lifestyle, or public service announcements—often prove ineffective in competing with the instant gratification offered by digital entertainment.

In this context, there is a need to seek new, interactive approaches to stimulating interest in sports. One such tool is gamification—the use of game mechanics, dynamics, and aesthetics to engage people, encourage action, and solve complex problems [1, 2]. Creating software products that simulate real-life processes—in particular, workouts at the gym—helps lower the “barrier to entry” into sports. By controlling a virtual character, the user gains a safe space to experiment, learn the basics of the training process, and, most importantly, see a clear visualization of the results of their efforts.

Developing a gym simulation game is relevant not only as an entertainment project but also as an educational and motivational tool. It demonstrates the logical connections between discipline, regular training, the proper allocation of resources (time and energy), and improvements in personal performance. Such a virtual hobby can be the first step

toward a real-life lifestyle transformation, since game mechanics (the system of achievements, levels, and visual character upgrades) operate on the principle of positive reinforcement, which can easily be applied to real-life practice.

From a technical standpoint, the relevance of this research stems from the need to apply modern software development methodologies to create complex interactive systems. The choice of the Java programming language for this project is strategically sound. Java is a high-level, object-oriented programming language that ensures cross-platform compatibility, reliability, and security of the software product.

Designing a game simulator requires a deep understanding of the object-oriented programming (OOP) paradigm. The structure of a sports hall can be perfectly decomposed into classes and objects: the entities “Coach,” “Athlete,” “Exercise Machine,” “Equipment,” and “Training Plan” each have their own attributes and interaction methods. Using Java allows us to implement the complex logic of how these objects interact, while ensuring adherence to fundamental programming principles such as SOLID, DRY (Don’t Repeat Yourself), and KISS (Keep It Simple, Stupid). This ensures the creation of clean, testable, and scalable code, which is critical for any professional developer.

Developing the graphical user interface and handling game events in Java allows the user to immerse themselves in an environment where every choice has consequences. Programming the logic of fatigue, energy recovery, and virtual muscle hypertrophy requires the use of mathematical models and algorithms, making this course project a comprehensive study at the intersection of motivational psychology, sports physiology, and computer science.

The goal of this course project is to provide a theoretical foundation, design, and practical implementation of an interactive gym simulation game written in the Java programming language, which, through gamification mechanisms, will promote sports and a healthy lifestyle among users.

To achieve this goal, the following tasks must be completed:

1. Analyze current trends in the development of game simulators and gamification methods as a means of promoting self-development.

2. To investigate the subject area (the structure of the training process) in order to create an appropriate game model.
3. Formulate requirements for the software product and develop a concept for game mechanics.
4. Design the application architecture by defining key classes, their properties, and interaction methods based on OOP principles.
5. Implement the simulator's logic using the Java programming language, including character progression and resource management systems.
6. Develop a graphical user interface that ensures the process is user-friendly and interactive.
7. Conduct comprehensive testing of the developed software to ensure it meets functional requirements.

The subject of this study is the process of creating game simulators as tools for modeling socially significant activities and as means of motivating users.

The subject of the study is the methods, algorithms, and software tools of the Java programming language, as well as the principles of object-oriented design applied to develop the Tornado Gym desktop simulation game.

That is why the topic chosen for this term paper is the development of a software system for modeling the following objects: "TORNADO GYM Fitness Center Simulator. Macro-objects: Front Desk & Intake, Cardio Area, Weights Area. Micro-objects: Athlete, Power Lifter, Gym Boss."

1 ANALYSIS OF THE CURRENT STATE OF THE ISSUE AND JUSTIFICATION OF THE PROJECT TASK

1.1 Subject Area

A gym is a complex social system where different categories of visitors—from beginners to experienced athletes—interact simultaneously. Each person comes to the gym with their own goals, fitness level, and behavioral characteristics. It is precisely this

diversity that makes a fitness center an interesting subject for modeling using object-oriented programming.

Modern society faces the problem of increasing physical inactivity—a chronic lack of physical activity. According to the World Health Organization (WHO, 2022)[3], more than 1.4 billion adults worldwide lead a sedentary lifestyle, which increases the risk of cardiovascular disease, type 2 diabetes, and depression. In Ukraine, according to sociological studies, only about 20–25% of young people engage in regular physical exercise.

The “Tornado GYM” simulator models a dynamic fitness center environment from a top-down view. It defines a three-level hierarchy of visitor micro-objects and three functional zones—macro-objects.

The hierarchy of micro-objects (athletes) is built on the principle of inheritance:

Athlete (base class).

A typical gym visitor, beginner, or someone working to stay in shape. Characteristics: moderate movement speed, standard workout duration, preference for the cardio zone. The class contains the following main fields: name (String name), stamina level (double stamina), current zone (String currentArea), speed (double speed). Graphical representation: a yellow circle with the name label and a stamina bar. The image is complemented by an elliptical shadow.

Power Lifter (subclass of Athlete).

An experienced athlete specializing in strength training. It inherits all fields and methods from the Athlete class and extends them: the fields `int maxWeight`` (maximum working weight) and `double recoveryRate`` (recovery rate) are added. Behavior: higher endurance, slower movement (greater “mass”), pronounced advantage in the Weights area. Power Lifter overrides the parent class’s `train()` method by calling `super.train()` for the base logic and adding strength exercises. Visually, it differs from Athlete in that it has a larger, blue-colored figure.

Gym Boss (a descendant of Power Lifter).

An elite member—the gym’s authority figure, at the highest level of the hierarchy. Inherits the fields from Power Lifter and adds: `int influenceRadius` (radius of influence on others), `boolean isMotivating` (indicator of active motivation). Gym Boss overrides the `interact()` method and calls `super.interact()` from Power Lifter. Unique ability: when within the influence zone of other athletes, it increases their stamina (`motivate()`). It has priority access to exercise machines. Graphical representation: a red figure with a golden halo and a crown symbol.

The inheritance hierarchy illustrates the key principles of OOP. Each subsequent level extends the previous one without duplicating code (the DRY principle). All three classes can be displayed on the screen and interact with each other—this demonstrates polymorphism: an array of type `List<Athlete>` can contain objects of all three types, and each will execute its own implementation of the `train()` method.

The differences between the classes in terms of inheritance are shown in Table 1.1.

Table 1.1—Comparison of Micro-Object Class Characteristics

Characteristic	Athlete	Power Lifter	Gym Boss
Hierarchy Level	1 (basic)	2 (subclass of Athlete)	3 (descendant of Power Lifter)
Speed	Average	Low (heavy)	Low + priority
Stamina	Standard	High	Maximum
Priority Zone	Cardio zone	Weights area	Weights area
Special Ability	None	Strength exercises	Motivate neighbors (motivate())
Shape color	Yellow	Blue	Red + gold halo
Additional fields	name, stamina, speed, currentZone	+ maxWeight, recoveryRate	+ influenceRadius, isMotivating

Gym zone macro-objects are containers for micro-objects. The “Tornado GYM” simulator defines three zones:

Front desk & Intake—the entrance area and reception desk.

This is where athletes appear at the start of the simulation and check in. The zone serves as the entry point: newly created athlete objects appear here and proceed from here to the training zones. This mirrors the real-life process of visiting a gym and adds authenticity to the simulation.

Cardio Area - the cardio zone.

It contains treadmills and stationary bikes. It is primarily geared toward entry-level athletes (Athlete). Athletes in this zone perform aerobic exercises, gradually improving their endurance.

Weights area. Contains strength training machines and free weights. This is the priority zone for Power Lifters and Gym Bosses. Athletes interact with one another: the presence of a Gym Boss increases the intensity of neighboring athletes' workouts.

All simulation takes place within the universal GymWorld object—a top-level container that unites all zones and all athletes. The program window displays only a portion of the GymWorld space (no more than 25% of the total size), creating the feeling of a large, lively gym.

1.2 Development of technical specifications for the project

In the Java programming language, you must create an application with a graphical user interface. The micro-object class must contain at least 4 variables and at least 4 methods (excluding getters and setters). At least one variable must be of type int, one must be of type double, and at least one must be a string. Microobject image: at least three graphical primitives, one of which must be text (even when inactive). A white (invisible) graphical primitive does not count. Add a reference type to the microobject class as an element to make deep copying necessary. Implement deep copying for the micro-object class. The micro-object's image must contain graphic primitives that reflect the state of the reference-type element added to the micro-object class to ensure deep copying is required during cloning. When the left mouse button is clicked on a microobject, it must become active or inactive. The program must support the simultaneous activation of multiple objects. The image of an active microobject must differ from that of an inactive one by at least one graphical primitive. The program interface must indicate which micro-object the user has selected. At the student's discretion, information about the active micro-object(s) may be displayed in the status bar or the program's workspace. If the user has selected multiple micro-objects, this information must also be displayed in some way. When the arrow keys are pressed, the active object(s) should move within the program window. When the Delete key is pressed, the active objects should be deleted. If there is no active object, the Delete key is ignored. The Esc key should cancel the activation of the object. A specific key combination (e.g., Ctrl-C) should be used to copy the active

micro-object. When the right mouse button is clicked on a micro-object, a dialog box should appear that allows you to change the parameters of that micro-object. Add three or more macro-objects to the project. Macro objects must be able to contain more than two micro objects at the same time. Add commands for drawing macro objects to the program. It is not necessary to include the ability to add or remove macro objects in the program. It is sufficient if the program has a certain number of macro objects generated at startup that remain unchanged throughout the program's operation. Macro-objects must have specific positions within the universal object and must be displayed on the screen. The positions of macro-objects within the universal object may remain unchanged throughout the program's execution. The universal object must contain at least one instance of each macro-object declared in this assignment. The image of each macro-object must consist of at least three graphic primitives (line, rectangle, circle, arc, polygon, text, graphic image, and others), and one of the primitives must be text. A white (=invisible) graphic primitive does not count. Micro-objects may belong to one or more macro-objects, or to none at all. It must be clear to the viewer how a micro-object belongs to a macro-object—that is, how the appearance (or behavior) of such a micro-object differs from that of a micro-object that does not belong to any macro-object. It must be possible to remove a micro-object from all macro-objects, resulting in it not belonging to any macro-object. It must be possible to make the active micro-object belong to one of the existing macro-objects. The display of a macro-object must show the number of micro-objects that belong to it. A hierarchy of micro-object classes must be constructed, containing at least three levels of inheritance that can be instantiated and displayed on the screen. The representation of each level must differ from all others by at least one graphical primitive. The constructed class hierarchy must demonstrate: a) calling the base class constructor; b) calling a base class method; c) using dynamic polymorphism; d) using static polymorphism. The use of the `instanceof` keyword must be demonstrated. Automatic movement of some micro-objects must be implemented: 1) during movement, some micro-objects interact with each other (this must be visually noticeable in the sense that either something changes in their appearance or in the nature of their movement); 2) some micro-objects enter macro-objects and some exit them; 3) the nature of the micro-

objects' movement must change when a specific keyboard key is pressed or when a menu command is selected. By "nature of movement," we mean not just the trajectory, but the way in which they interact with macro-objects and other micro-objects. The program window displays only a specific portion of the game world, which must not exceed 25% of its total size. Implement the ability to change the portion of the game world that the user sees (by any means). A minimap must be implemented to allow for quick navigation: 1) the visible area of the universal object must be marked on the minimap; 2) the movement of the visible area of the universal object updates the minimap accordingly; 3) macro-objects and micro-objects are marked on the minimap; 4) the movement and interaction of micro-objects result in corresponding changes on the minimap; 5) clicking the left mouse button on the minimap moves the visible area of the universal object. Serialization/deserialization of all objects into a text/binary/XML file must be implemented. Serialization/deserialization must preserve not only the state information of macro- and micro-objects but also their positions on the map. During serialization/deserialization, dialog boxes must be used to prompt the user for the file name and its location on the disk (for example, the system FileChooser dialog box can be used). Demonstrate the use of a lambda expression. Demonstrate the use of a Stream operation. When the Insert key is pressed, a dialog box should appear to define the parameters of the micro-object being created. It must include the option to select which of the derived classes in the inheritance hierarchy it belongs to. In addition to the Button control, the dialog box must also include at least three of the following: TextField, CheckBox, {ComboBox || ListView || ChoiceBox}, RadioButton.

The following query must be implemented: find a micro-object with user-specified parameters, as a result of which the program searches for the micro-object and displays on the screen: its location and whether this micro-object belongs to any macro-objects (or indicates that the micro-object does not belong to any macro-object). The following query must be implemented: display a list of micro-objects that belong to the specified macro-object. The following query must be implemented: display a list of micro-objects that do not belong to any macro-object. The following query must be implemented: set the sorting criteria for micro-objects in queries that return lists. There must be at least three sorting

criteria, for example: Student_Name, Average_Score, Number_of_Grades_of_4_or_Higher.

At least three queries for counting micro-objects must be implemented. For example: a) the number of active micro-objects, the number of micro-objects with a life level greater than half, the number of micro-objects located in the right half of the screen, and so on.

The following hardware is required for the program to run properly:

1. An IBM PC-compatible computer with a clock speed of 2200 MHz or higher.
2. Operating systems: Windows Vista/7/10; Linux; Mac OS.
3. 8 GB of DDR4 RAM.
4. An SVGA (Super Video Graphic Adapter) graphics card.
5. A video card with at least 2 GB of memory.
6. IBM keyboard and mouse.

Disk space required for the program to run properly: 25 Mb.

Amount of RAM used by the program: 270 Mb.

1.3 Rationale for the Choice of Programming Language

To accomplish the task at hand, the cross-platform programming language Java was chosen in combination with the JavaFX platform and the IntelliJ IDEA development environment [4, 5]. The main competitors in this selection were C++ and C#. Let's examine why Java is the optimal choice for implementing the "Tornado GYM" simulator.

Full support for the OOP paradigm. Java is one of the few languages where the object-oriented paradigm is not only supported but also fundamental. All language constructs are built around classes and objects. This allows for the natural implementation of a three-level hierarchy of athletes (Athlete → Power Lifter → Gym Boss), polymorphic behavior, and encapsulation without architectural compromises.

Cross-platform compatibility (WORA). Thanks to the "write once, run anywhere" principle, a Java program runs on any operating system where the JVM is installed. This

increases the accessibility of “Tornado GYM” for potential users regardless of their platform, whereas C++, for example, requires recompilation for each platform.

JavaFX as a modern GUI framework. The JavaFX platform provides the necessary tools[6, 7] for the simulator: the Canvas API for freely drawing objects on the game field, AnimationTimer and Timeline for implementing the game loop with smooth animation, and support for keyboard and mouse events. Unlike the outdated Swing, JavaFX uses hardware graphics acceleration via OpenGL/DirectX.

Built-in serialization support. The standard Java library includes a mechanism for serializing objects (the Serializable interface)[8, 9], as well as tools for working with XML and JSON. This simplifies the implementation of saving and loading the complete state of the “Tornado GYM” simulation without requiring external libraries.

A list of features unique to Java compared to its competitors:

1. Java is a fully cross-platform general-purpose language. Programs written in it run on virtually everything—from supercomputers to microchips—whereas C++ is designed from the outset to be compiled for a specific platform.
2. Java offers extensive multithreading capabilities, allowing the “Tornado GYM” game loop and the serialization subsystem to run in parallel.
3. Java is fully backward-compatible with previous versions—projects created several years ago will work perfectly today.
4. The concept of OOP is much better implemented in Java than in other programming languages. Java handles all low-level memory management: the built-in garbage collector takes care of removing unnecessary objects.

List of Java’s drawbacks compared to C++ and C#:

1. Java pays for its cross-platform capability with performance: code written in C++ runs faster and requires fewer computer resources.
2. Java lacks the operator overloading found in C#, which can make the code less understandable in some non-standard situations.
3. Java does not provide the ability to create generators, a feature found in C#, JavaScript, and Python.

Table 1.2 - Comparison of Programming Languages

Features	C++	C#	Java
Paradigm	Multiparadigm (includes OOP and procedural programming), which sometimes complicates the architecture.	OOP and component-oriented, tightly integrated with the Microsoft ecosystem.	A benchmark for OOP, ideally structured for creating scalable and easily maintainable solutions.

Continuation of Table 1.2

Features	C++	C#	Java
Year of release	1983 (has a lot of outdated "legacy" approaches).	2000 (developed as Microsoft's response to Java's success).	1995 (time-tested, has the most stable evolutionary path).
Execution mechanism	Direct compilation to machine-dependent code.	Compilation to intermediate code (IL), primarily geared toward the .NET environment.	Bytecode + JVM, providing unique flexibility, hardware independence, and dynamic optimization.
Scope of Application	Highly specialized tasks: game development, drivers, high-load system modules.	Development of enterprise solutions, primarily for Windows, and games (Unity).	Versatility: the undisputed leader in the enterprise sector, cloud services, banking systems, and Android development.
Syntax and readability	Overloaded and difficult to grasp, with a high learning curve.	Modern, but in some places overloaded with "syntactic sugar."	Concise, rigorous, and intuitive, making the code as readable as possible.
Cross-platform compatibility	Low. Requires separate	Good, but historically works best in the	Absolute ("Write Once, Run Anywhere"). Runs

	recompilation for each operating system.	Windows environment.	anywhere the Java Virtual Machine is installed.
--	--	----------------------	---

Continuation of Table 1.2

Features	C++	C#	Java
Cross-platform compatibility	Low. Requires separate recompilation for each operating system.	Good, but historically works best in the Windows environment.	Absolute (“Write Once, Run Anywhere”). Runs anywhere the Java Virtual Machine is installed.
Performance	Maximum, but at the cost of increased development complexity and the risk of errors.	High, optimized for the .NET infrastructure.	High and stable. Modern JIT compilers bridge the performance gap, providing an ideal balance of performance and reliability.
Security and Memory	Manual memory management (constant risk of memory leaks and critical vulnerabilities).	Automatic garbage collection, reliable architecture within .NET.	Unprecedented level of security. Isolated execution in the JVM and advanced garbage collector algorithms prevent most fatal errors.
Ecosystem (Extensibility)	Fragmented, with no single standard for package management.	Powerful, but heavily centralized around Microsoft tools and paid solutions.	The world’s largest open-source community. A vast selection of free libraries and reference frameworks covering both desktop and server solutions[10].

Thus, taking into account the results of a multi-criteria comparison of object-oriented programming languages (see Table 1.2), we can conclude that the Java programming language, combined with the JavaFX platform and the IntelliJ IDEA environment, is the optimal choice for implementing the given task.

1.4 Conclusions

Thus, in the first chapter, a comprehensive analysis of the subject area was performed, and the problem statement for the course project was justified.

The subject domain of the “Tornado GYM” simulator was defined as a dynamic gym environment viewed from above. A three-level hierarchy of athlete micro-objects was described: the base class Athlete (a regular visitor), the Power Lifter class (a subclass of Athlete, a strength athlete), and the Gym Boss class (a subclass of Power Lifter, an elite gym authority figure). A comparison table of these classes is provided, illustrating the principle of inheritance and functionality extension: each level of the hierarchy adds new fields and overrides methods of the parent class, demonstrating polymorphism. The Gym Boss, in particular, has the unique ability to motivate nearby athletes via the `motivate()` method, which is an example of dynamic polymorphism.

Three zone macro-objects are described: Front desk & Intake (entrance area), Cardio area (cardio zone for Athlete), and Weights area (strength training zone for Power Lifter and Gym Boss). All of them operate within the GymWorld container.

Technical specifications for the project have been developed: the functional and non-functional requirements for the program, as well as the hardware and software requirements for the system, have been defined. The choice of the Java programming language and the JavaFX platform was justified based on the following criteria: full support for OOP, cross-platform compatibility, a modern GPU-accelerated GUI, built-in serialization, and industry demand. A comparative analysis of Java, C++, and C# is presented in Table 1.2, confirming that the chosen language is optimal for implementing the simulator.

2 DEVELOPMENT OF THE GRAPHICS DISPLAY SUBSYSTEM

2.1 Graphics Rendering Model

Each athlete micro-object consists of several JavaFX graphical elements that form a single visual group. All elements are positioned relative to two base coordinates, X and Y, which are private fields of the Athlete class. The offset values are defined directly in the code of the `initVisuals()` method.

The “Tornado GYM” simulator defines three types of athlete micro-objects that form a three-level inheritance hierarchy. Each type has its own graphical representation.

Athlete (base class) — a regular gym-goer. Depicted as a slender figure wearing a black T-shirt with the “Tornado GYM” logo and black shorts. The graphical representation includes: a PNG image of the character (`imageView`), a text block with the name (`nameText`) and the name of the equipment (`equipmentText`), a yellow stamina bar (`energyBar`), and a rainbow-colored activation border (`activeBorder`). An image of the Athlete micro-object is shown in Figure 2.1, a.

Power Lifter (a subclass of Athlete) is an experienced strength athlete. It is depicted as a muscular figure in a blue weightlifting jumpsuit with the “Tornado GYM” logo. The graphical representation is structurally identical to that of Athlete but uses a different, enlarged PNG image of the character. An image of the Power Lifter micro-object is shown in Figure 2.1, b.

Gym Boss (a descendant of Power Lifter) is the gym’s elite authority figure, representing the highest level of the hierarchy. He is depicted as a massive figure in a red tracksuit. The graphical representation is structurally identical but uses its own PNG image. The image of the Gym Boss micro-object is shown in Figure 2.1, c.



Figure 2.1 – Images of micro-objects in the program: a) Athlete; b) Power Lifter; c) Gym Boss

The coordinates of all constituent elements of the Athlete micro-object are calculated relative to the base X and Y coordinates, as clearly shown in Figure 2.1. The corresponding coordinate values for each element are given in Table 2.1.

Table 2.1 – Coordinates of the Athlete micro-object's elements relative to the base point (X, Y)

Element	x	y	width × height
textBg (Rectangle)	X + 2	Y + 2	156 × 40
nameText (Text)	X + 5	Y + 18	—
equipmentText (Text)	X + 5	Y + 33	—
energyBarBg (Rectangle)	X + 5	Y + 40	150 × 8

Continuation of Table 2.1

Element	x	y	width × height
energyBar (Rectangle)	$X + 5$	$Y + 40$	$150 \times (\text{energyLvl}/100) \times 8$
zoneEmojiView (ImageView)	$X + 138$	$Y + 4$	22×22
imageView (ImageView)	$X + 10$	$Y + 50$	140×210
activeBorder (Rectangle)	$X + 10$	$Y + 50$	140×210

Each zone macro-object is a container for micro-objects and consists of several JavaFX graphical elements that form a single group. All elements are positioned relative to the zone's base X and Y coordinates and its width and height parameters. The “Tornado GYM” simulator defines three macro-object zones.

Front Desk & Intake — the entrance area and reception desk of the gym. It consists of an administrative counter with an illuminated “Tornado GYM” logo, shelves with dietary supplements, and informational monitors. The macro-object includes: a PNG image of the reception desk (width = 600, height = 280), a themed emoji icon before the zone name, a text label with the zone name, and an athlete counter. An image of the “Front Desk & Intake” macro-object is shown in Figure 2.2.



Figure 2.2 - Image of the “Front desk & Intake” macroobject

Cardio area — cardio zone. It features rows of treadmills with touchscreen monitors, elliptical trainers, and stationary bikes bearing the “Tornado GYM” logo. The macroobject includes: a PNG image of the zone (width = 480, height = 320), a themed emoji icon before the zone’s name, a text label with the name, and an athlete counter. An image of the “Cardio area” macroobject is shown in Figure 2.3.

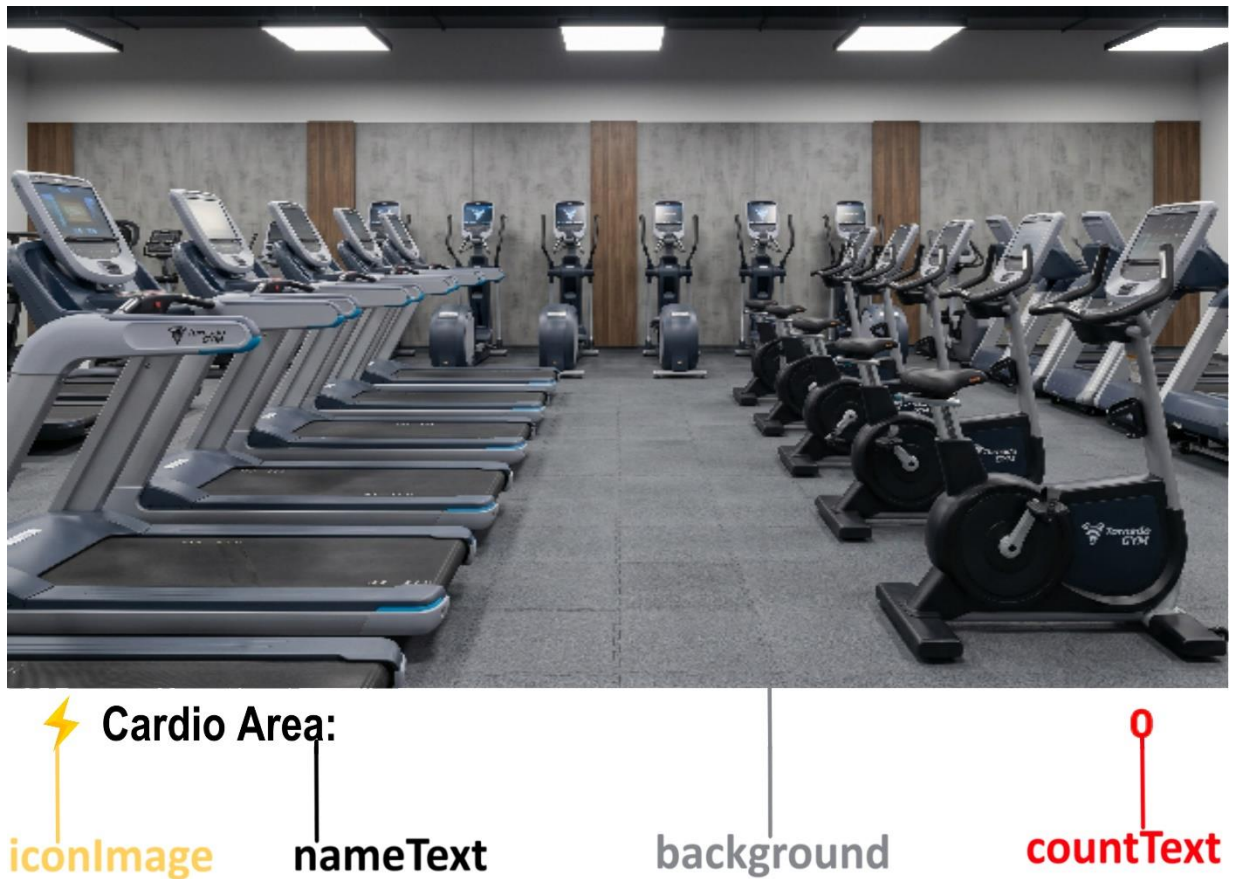


Figure 2.3 - Image of the “Cardio area” macroobject

Weights area — the weightlifting zone. It contains power racks with barbells, dumbbells, and weight plates featuring the “Tornado GYM” logo, racks with dumbbells of various weights, and incline benches. The macroobject includes: a PNG image of the area (width = 480, height = 320), a themed emoji icon before the area’s name, a text label with the name, and an athlete counter. An image of the “Weights area” macroobject is shown in Figure 2.4.

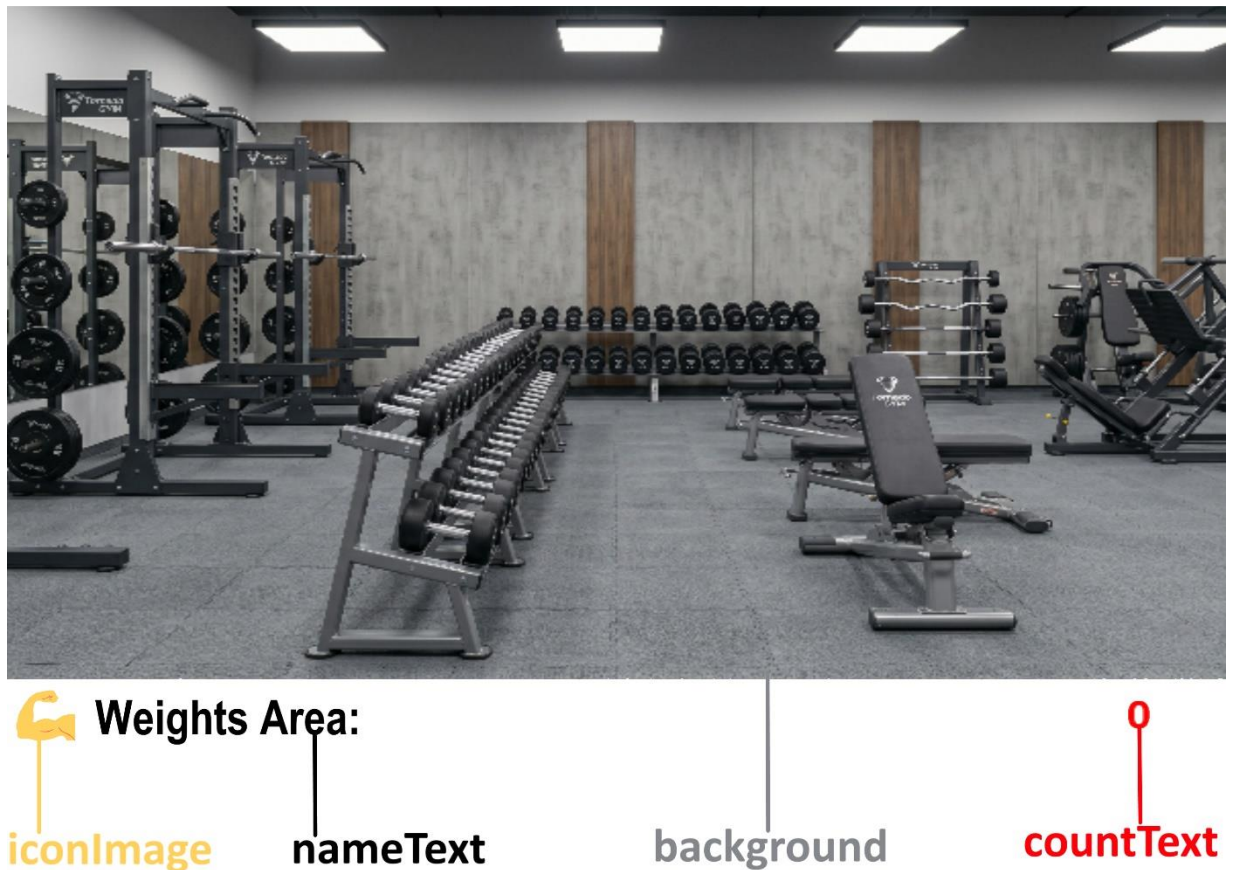


Figure 2.4 - Image of the “Weights area” macroobject

The coordinates of all constituent elements of the “TrainingZone” macro-object are calculated relative to the base coordinates X , Y and the zone’s width and height parameters, as clearly shown in Figures 2.2–2.4. The corresponding coordinate values for each element are listed in Table 2.2.

Table 2.2 – Coordinates of the TrainingZone macro-object elements relative to the base point (X , Y)

Element	x	y
background (ImageView)	X	Y
iconImage (ImageView)	$X + 5$	$Y + \text{height} + 5$
nameText (Text)	$X + 35$	$Y + \text{height} + 23$
countText (Text)	$X + \text{width} - 40$	$Y + \text{height} + 23$

All macro-objects dynamically display the number of micro-objects contained within them at any given moment during the simulation. The GymWorld container combines all three zones and all athletes, while the program window displays only a portion of its space, creating the feeling of a large, lively gym.

2.2 Graphic Procedures of the Graphics Rendering Subsystem

In the program, the graphical rendering of micro- and macro-objects is implemented using the JavaFX library [4, 6]. Below is a list of key graphical primitives and methods used during development.

`ImageView(Image image)` — a container element for displaying raster images on the JavaFX stage, derived from the `Node` class. Used to display PNG sprites of characters and game zones.

`Image(String string)` — loads a raster image from the specified path in the file system or from the application's resource directory (`/resources/images/`).

`Label(String string)` — a text interface element with specified content. It is used to display the athlete's name above their sprite, as well as to display the zone name and the current number of athletes in the macro-object.

`Rectangle(double width, double height)` — a rectangular shape derived from the `Shape` class with specified dimensions. It is used when constructing the stamina bar: a static gray rectangle serves as the background, and a dynamic rectangle is placed on top of it, whose width changes proportionally to the current value of the stamina field.

`setEffect(Effect value)` — a method for setting a visual effect for any scene node. In this project, it is used to apply a `GaussianBlur` effect to the characters' elliptical shadows, which adds realism to the image.

`setRotate(double value)` — sets the rotation angle of an element relative to its central axis. It is used to slightly tilt the shadows beneath the athletes to create the illusion of spatial depth.

`setOpacity(double value)` — adjusts the node's opacity level within the range of 0.0–1.0. It is used for shadows (values between 0.35 and 0.5) and during animations where objects appear and disappear on the scene.

`Group(Node... children)` — a container that groups any number of Node objects for their collective placement and control on the main screen. Each micro- and macro-object is encapsulated in its own group, which simplifies their movement and control over the z-order of display.

`setOnMouseClicked(EventHandler<MouseEvent> value)` — registers a click event handler for a node. It is responsible for switching the athlete to manual control mode: clicking the left mouse button on the character's image activates or deactivates this mode.

`AnimationTimer()` — the foundation of the game loop, containing the methods `start()`, `stop()`, and `handle(long now)`, which is called continuously on every frame. It is through `AnimationTimer` that the “Tornado GYM” game loop is implemented: in the `handle()` method, the athletes' positions are calculated frame by frame, the status of the stamina indicators is updated, collisions with zone boundaries and between objects are checked, and then the scene is redrawn.

`Timeline` and `KeyFrame` are tools for creating smooth, frame-by-frame animations of specific events: the appearance of a new character (fade-in effect), the activation of the `motivate()` ability in the Gym Boss (a golden flash around the character), and an athlete's transition between zones.

`Media(String url)` and `MediaPlayer(Media media)` — handle the loading and playback of MP3 audio files within the application. The audio track includes background music for the gym and sound effects that accompany athletes' interactions with the exercise machines.

3 DEVELOPMENT OF CLASS, OBJECT, AND STATE DIAGRAMS

A class diagram is a static representation of a model's structure[11]. It depicts static (declarative) elements, such as classes, data types, their contents, and relationships. This section discusses the main types of UML diagrams that depict the architecture of the “Tornado GYM” gym simulator: the hierarchy of micro-objects, relationships between classes, the sequence of interactions, and the behavior of objects in different states.

3.1 Inheritance Diagram

Inheritance is one of the fundamental principles of object-oriented programming, which allows you to create new classes based on existing ones, inheriting their properties and methods. This helps avoid code duplication and aligns with the DRY (Don't Repeat Yourself) principle [12, 13]. The developed program implements a hierarchy of micro-objects (Figure 3.1).

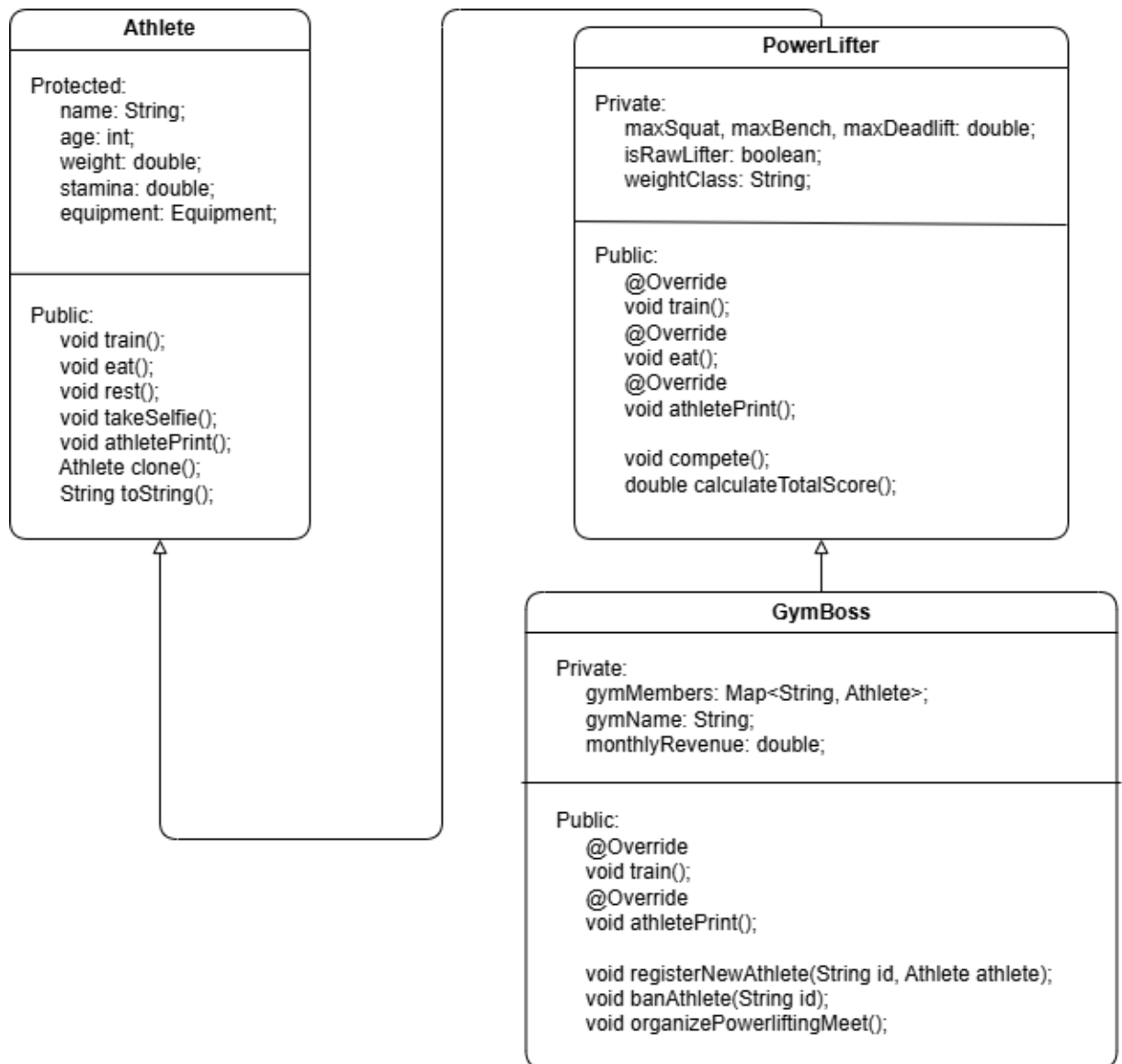


Figure 3.1 - Inheritance Diagram

The base class is `Athlete`, which contains fields common to all gym visitors: name, age, weight, stamina, and a reference to the equipment object. The base class's main methods include: `train()`, `eat()`, `rest()`, `takeSelfie()`, and `athletePrint()`.

The derived class `PowerLifter` extends the functionality of the base class by adding fields specific to strength sports (`maxSquat`, `maxBench`, `maxDeadlift`, `isRawLifter`, `weightClass`) and methods (`compete()`, `calculateTotalScore()`). It also overrides the `train()`, `eat()`, and `athletePrint()` methods to implement polymorphism.

Another class in the hierarchy is `GymBoss`. It is important to note that within the structure of this software system, `GymBoss` acts specifically as a full-fledged micro-

object (a character on the map), rather than as a macro-object or system manager. It contains unique fields (`gymMembers`, `gymName`, `monthlyRevenue`) and specific methods for managing the gym (`registerNewAthlete()`, `organizePowerliftingMeet()`), while polymorphically overriding the base training methods.

3.2 Aggregation Diagram

Aggregation represents a “part-whole” relationship, where the part can exist independently of the whole. If the container object is destroyed, the member object continues to exist in the system. In the “Tornado GYM” simulator, aggregation is demonstrated by the relationship between the `Athlete` and `Equipment` classes (Figure 3.2).

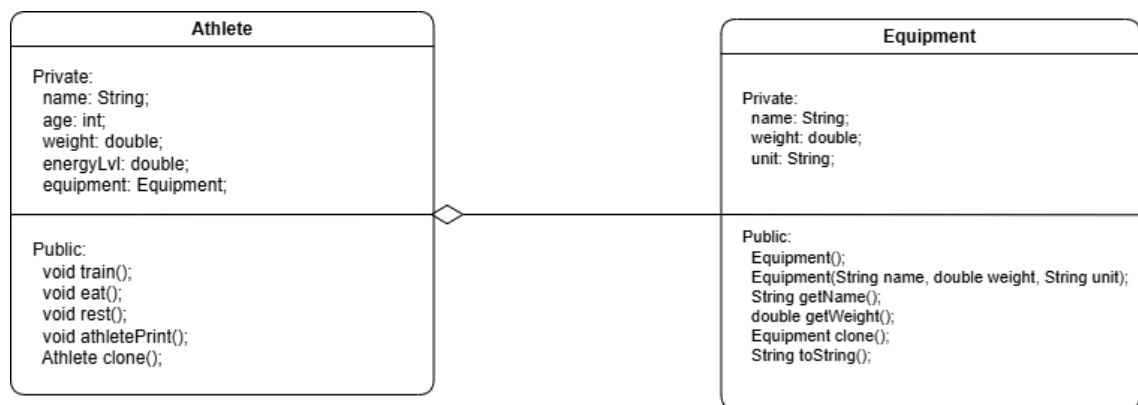


Figure 3.2 - Aggregation Diagrams

Each athlete micro-object can have specific equipment (for example, a weightlifting belt or a fitness band), a reference to which is stored in the equipment field. However, the Equipment object is completely independent: if the Athlete object is removed from memory (for example, the character leaves the gym), the equipment itself may still exist or be transferred to another micro-object.

3.3 Composition Diagram

Composition is a stricter form of aggregation in which the lifecycle of the parts is entirely dependent on the lifecycle of the container. A part object cannot exist without the

whole object. In the program, composition is clearly evident in the creation of the graphical interface for training zones. The TrainingZone class consists of a series of graphical primitives: background and iconImage (of type ImageView), nameText and countText (of type Text), border (of type Rectangle), and visualGroup (of type Group) (Figure 3.3).

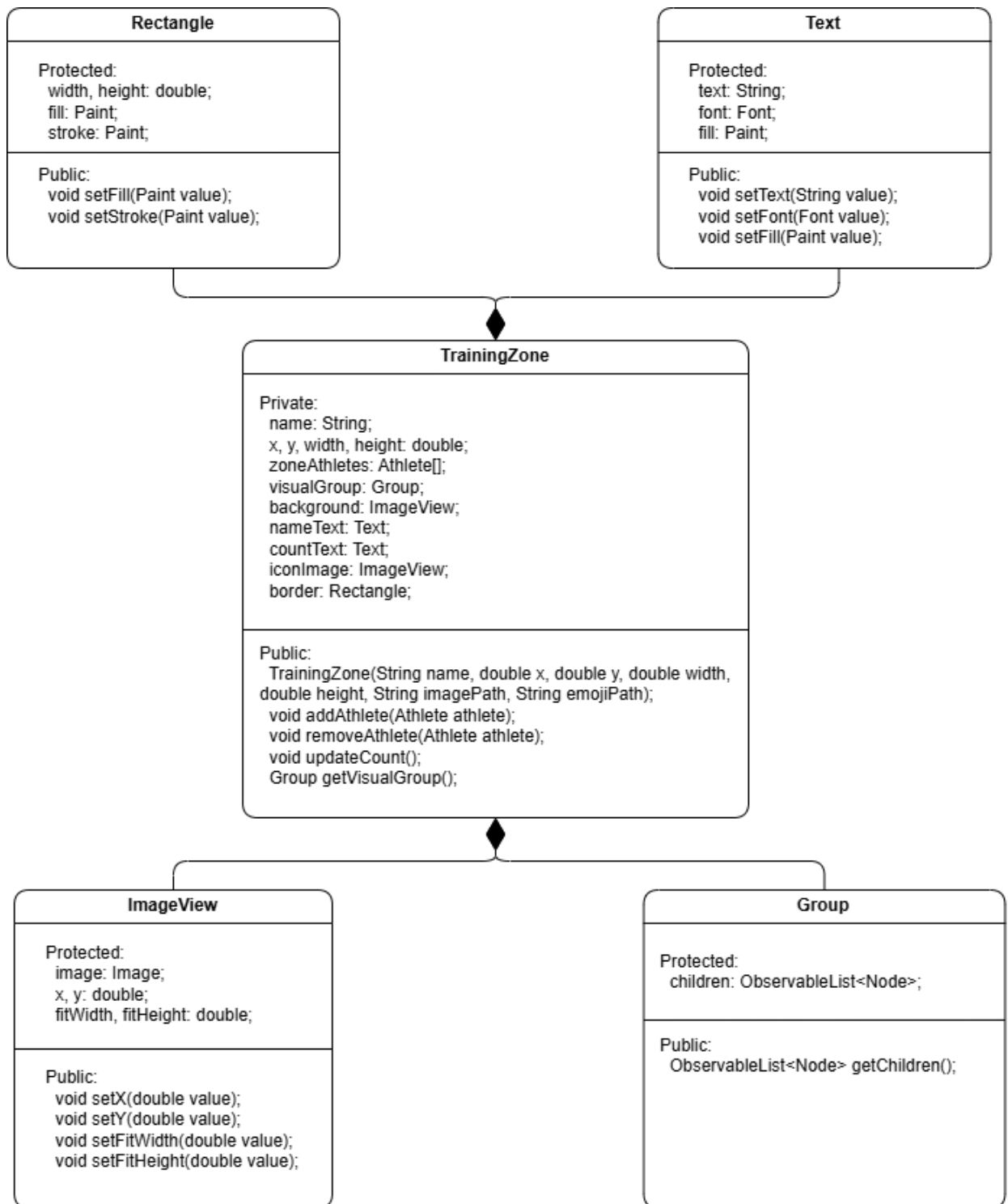


Figure 3.3 - Composition Diagram

These graphical elements are initialized and created directly within the TrainingZone class, forming its visual representation on the screen. If a training zone object is deleted, all of its constituent graphical components will also be automatically destroyed by the garbage collector, since they have no logical reason to exist separately.

3.4 Association Diagram

An association is a relationship between two classes that allows one object to interact with another to exchange messages or data. Figure 3.4 shows an association diagram between the Athlete micro-object and the TrainingZone object.

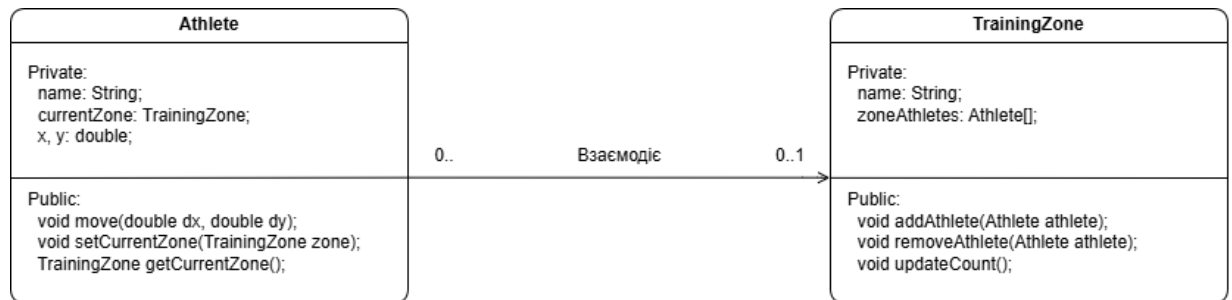


Figure 3.4 - Association Diagram

The Athlete object has a `currentZone` field that stores a reference to the zone in which the athlete is currently located (or the value null if the athlete is outside any zones). In turn, the TrainingZone object contains a `zoneAthletes` array that records all athletes currently training in that zone. Thanks to this bidirectional association, when an athlete moves across the scene, the system checks for coordinate intersections (using the ``intersects()`` method) and calls the corresponding ``addAthlete()`` or ``removeAthlete()`` methods to facilitate interaction. This ensures that the number of visitors in a specific zone is accurately displayed on the screen and that the athlete's status is updated.

3.5 Cooperation Diagram

A communication diagram is a type of interaction diagram that shows the relationships between objects, focusing on their structural organization and the messages they exchange. Unlike a sequence diagram, the emphasis here is on the architectural relationships between objects.

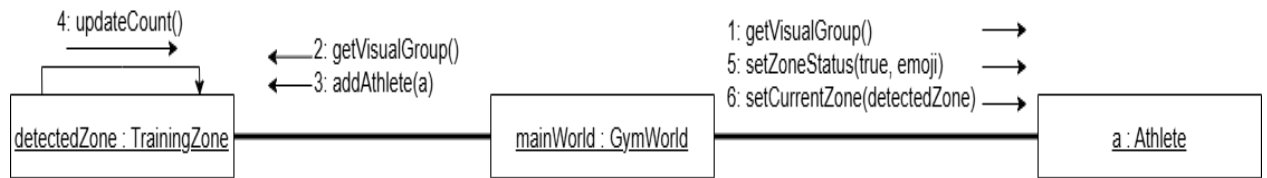


Figure 3.5 - Communication Diagram

In the “Tornado GYM” simulator, the cooperation diagram (see Figure 3.5) illustrates the interaction between the program’s main class, `GymWorld` (which contains the `AnimationTimer` game loop), the `Athlete` micro-object, and the `TrainingZone` object. The cooperation process is based on the transmission of signals as the character moves. The `GymWorld` object initiates the coordinate check. It sends a request to `Athlete` to obtain its current spatial boundaries using a graphical node. Upon detecting an intersection of the boundaries with the training zone, `GymWorld` passes the `Athlete` object as a parameter to the `addAthlete(athlete)` method of the `TrainingZone` class. Upon receiving the message, the `TrainingZone` object modifies its internal `zoneAthletes` array and calls the internal `updateCount()` method to update the text indicator on the screen.

3.6 Sequence Diagram

A sequence diagram depicts the interaction of objects in a system, ordered by time. It shows the lifelines of objects and the sequence of method calls in chronological order.

Figure 3.6 shows a sequence diagram of the process of registering and changing an athlete’s status upon entering a training zone. The initial event is the triggering of the `AnimationTimer` game timer in the `GymWorld` class. The sequence of actions is as follows:

1. `GymWorld` calls the `getVisualGroup().getBoundsInParent()` method on each `Athlete` and `TrainingZone` object to check for collisions.
2. If the `intersects()` method returns `true`, `GymWorld` sends a `removeAthlete(this)` message to the athlete’s previous zone (if one was specified).
3. `GymWorld` calls the `addAthlete(a)` method on the new `TrainingZone` object.

4. The TrainingZone class internally executes the updateCount() method, which changes the color and value of the countText counter.

5. GymWorld calls the setZoneStatus(true, emoji) and setCurrentZone(detectedZone) methods on the Athlete object, which causes the corresponding graphic icon to appear above the character and updates its internal state.

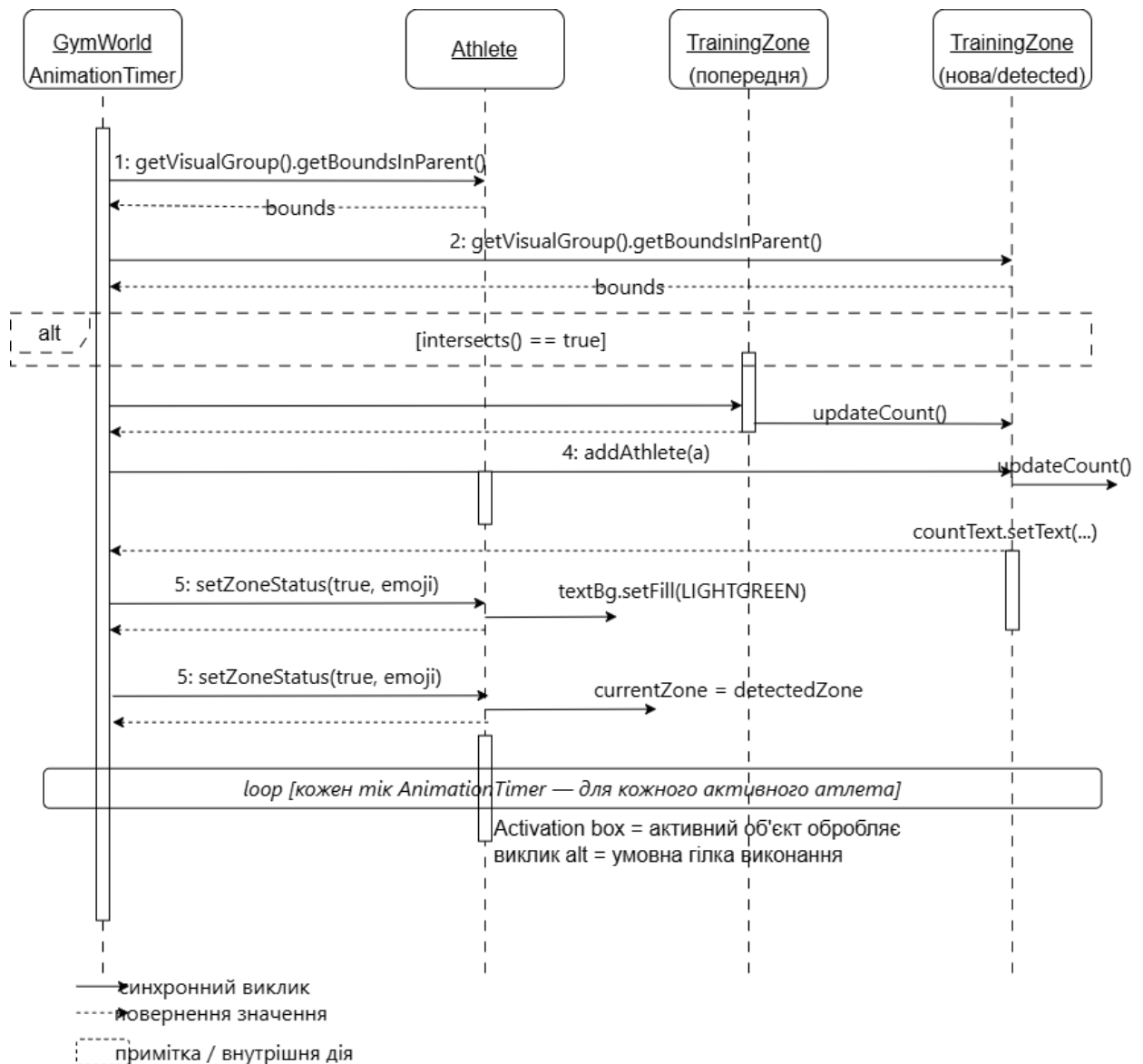


Figure 3.6 - Sequence Diagram

Thus, the described sequence of interactions ensures continuous synchronization of the athletes' spatial locations with the corresponding training zones and timely updating of the system's graphical interface in real time.

3.7 State Machine Diagram

A state machine diagram describes the dynamic behavior of a single object as a finite state machine. It records all possible states that an object can be in during its lifecycle, as well as the events and conditions that trigger a transition from one state to another.

A state machine diagram has been developed for the Athlete micro-object (Figure 3.7), which is based on changes in its coordinates and energy level (energyLvl). The following main states of the object have been identified:

- **Idle (Waiting / Rest):** The athlete's initial state upon creation on the map, when isActive = false and the coordinates do not change.
- **Moving:** The transition to this state occurs when control keys are pressed (event generation in 'activeKeys'). The character's texture moves across the scene using the 'move()' method.
- **Training:** This state is activated when the object is within the boundaries of the TrainingZone. In this state, a timer for reducing energyLvl (stamina) starts, and the energyBar graphic element is updated.
- **Exhausted / Resting:** An automatic transition to this state occurs when energyLvl falls below a critical threshold. The object disables the ability to train and executes the rest() or eat() methods (to restore energy), after which it returns to the resting state.

Діаграма стану: Athlete

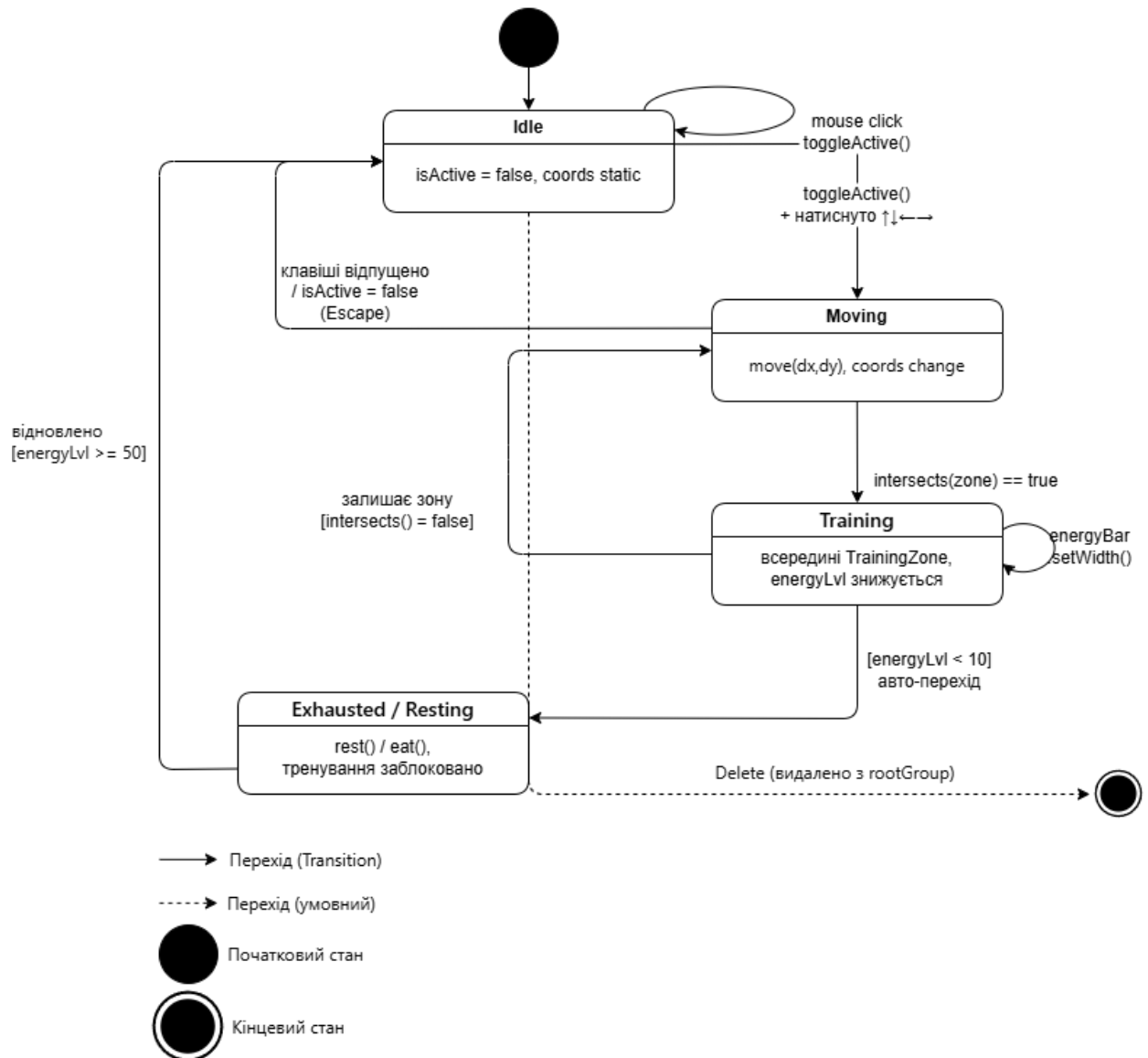


Figure 3.7 - State Diagram

Thus, the developed diagram fully describes the life cycle of the Athlete object, detailing its responses to user actions, spatial interaction with the game environment, and management of internal resources.

4 USER MANUAL

The GymWorld program is implemented as an executable JAR archive and does not require a traditional installation procedure. To get started, simply copy the GymWorld.jar file to any convenient directory on your computer and run it by double-clicking the left mouse button. The operating system will automatically hand control over to the installed JVM, which will execute the archive.

A prerequisite for running the program is having Java Runtime Environment (JRE) version 17 or higher installed. If only the JDK is installed on your computer, the program can still run. You can check your current Java version using the ``java -version`` command in the console (terminal/command line). Without Java, the program will not run, even as a compiled JAR file.

An alternative way to run the program is directly from the console: navigate to the directory containing the file and run the command ``java -jar GymWorld.jar``. If you have compiled .class files, make sure the path to the Java binaries is added to the PATH environment variable, then run the command ``java gymworld.GymWorld`` from the project's root directory.

4.1 Main Program Window

After successful launch, the main window opens at a resolution of 1920×1080 pixels, displayed in full-screen mode. The window displays a two-dimensional game map of the GymWorld sports hall with athletes moving around within it. In the upper-left corner is a mini-map that displays the current positions of all objects to scale.

The main part of the screen is occupied by the game area, which consists of three functional zones: the Front Desk, the Cardio Area, and the Weights Area. Each zone has its own background image and displays the number of athletes currently within its boundaries. The current count is displayed in the zone's label: in red if the zone is empty, and in black if athletes are present.



Figure 4.1 – Program interface upon successful launch

In the lower-left part of the window (see Figure 4.1) is a row of hotkeys displaying the available program control commands: move the camera using the Arrow keys or WASD, add a new athlete (INSERT), search (F), queries (G), automatic movement (P), select all objects (B), clone (Ctrl+C), delete the active athlete (DEL), save game state (Ctrl+S), load (Ctrl+L), and deselect (ESC). This bar serves as a constant visual cue for the user and eliminates the need to consult a separate help section.

4.2 Adding a New Athlete

There are two ways to add a new athlete to the gym: press the INSERT hotkey on the keyboard. A modal dialog box for creating a new character will open.

In the window (Figure 4.2), you must fill in the following fields:

1. Type—defines the athlete's class: regular Athlete, PowerLifter, or GymBoss. The character's appearance and behavioral traits depend on the type.
2. Name—a text string with the athlete's name, which will be displayed above their sprite.

3. **Weight (kg)** — the numerical value of the athlete’s weight. The default value is 80 kg.
4. **Equipment** — the “Has Equipment” checkbox allows you to specify the athlete’s equipment. If the checkbox is selected, choose a specific type of equipment from the drop-down list: dumbbells, barbell, jump rope, exercise machine, etc.
5. **Spawn Zone**—the location where the athlete will appear on the map: the reception area, cardio area, or weight training area.

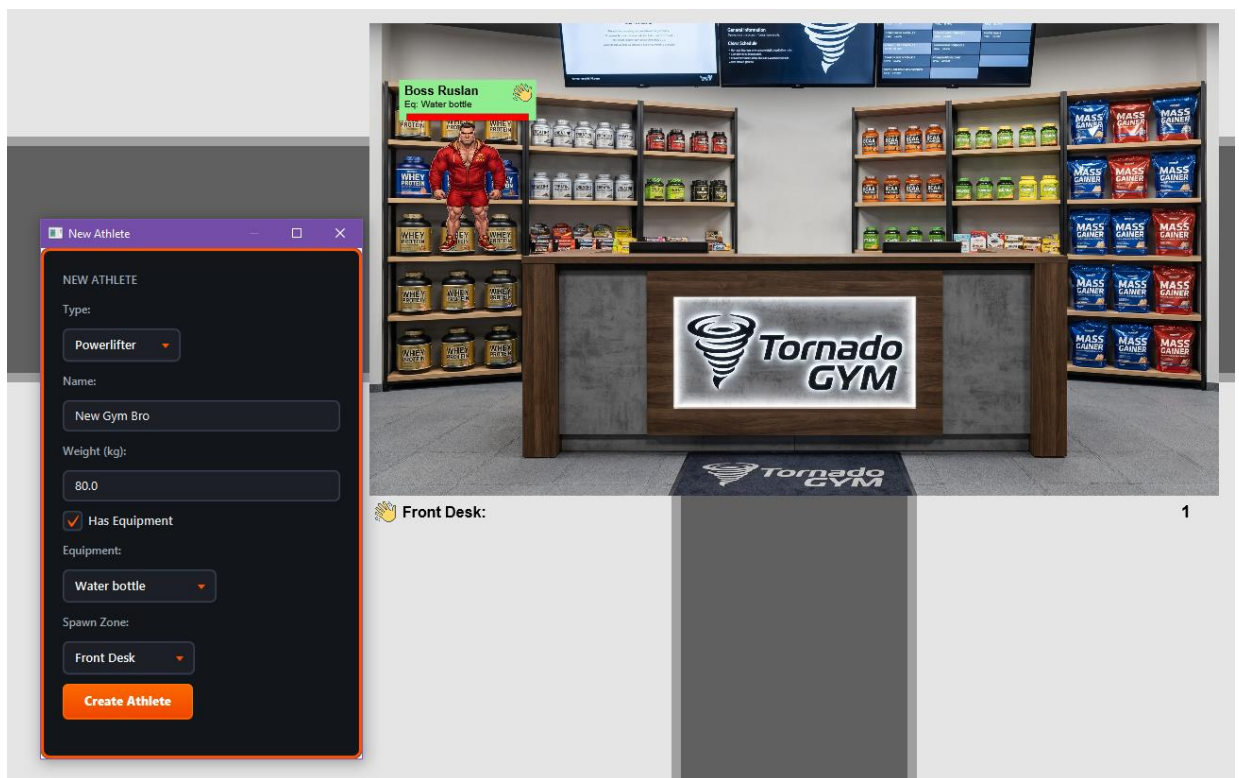


Figure 4.2 – Interface for adding a new athlete

After filling in all the fields, click the “Create Athlete” button. The new athlete will be placed in the selected zone and will immediately become visible on the playing field.

4.3 Moving Athletes

To start controlling an athlete, you must first make them active by left-clicking on them. An active athlete is highlighted by an animated rainbow frame around their sprite. You can have multiple active characters at the same time—they will all move simultaneously.

Movement is controlled using the WASD keys or the arrow keys:

- W / ↑ — move up;
- S / ↓ — move down;
- A / ← — move left;
- D / → — move right.

When an athlete enters one of the training zones, the background color of their info panel changes to light green, and an emoji icon corresponding to the current zone appears next to their name. At the same time, the athlete counter in the zone's caption increases. When the athlete leaves the zone, all changes are reversed.

In addition to manual control, athletes can use the automatic movement mode. In this mode, the character moves independently along a predetermined path, bouncing off the boundaries of the playing field. You can turn automatic movement on or off by pressing the P key. You can prevent the character from leaving the zone by pressing the B key.

4.4 Editing an Athlete

To edit the characteristics of an existing athlete, right-click on the athlete. An editing dialog box will open (Figure 4.3), containing the same fields as the creation window, but the “Type” field is locked and cannot be changed—the athlete's type is determined upon creation and cannot be changed.

The following fields are available in the editing window: name, weight, energy level (from 1 to 100), and equipment. The Energy Level affects the length of the colored

bar at the top of the athlete's sprite: a full bar represents 100 units, and an empty bar represents 1. Changes take effect immediately after clicking the “Save Changes” button.

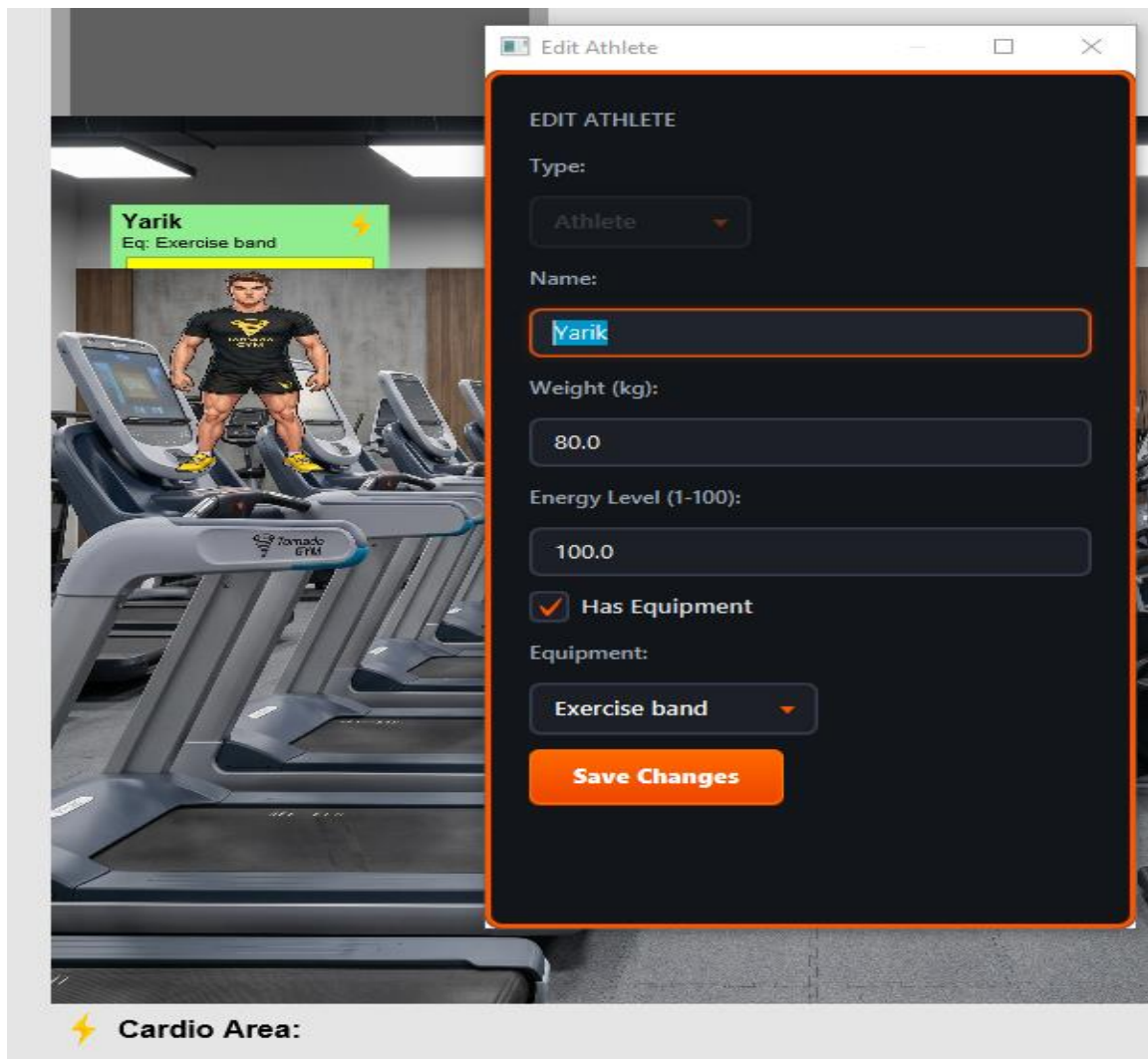


Figure 4.3 – Athlete Characteristics Editing Window

Thus, this functionality allows you to dynamically update an object's current parameters in real time, while strictly preserving its base typology, which is established at the creation stage.

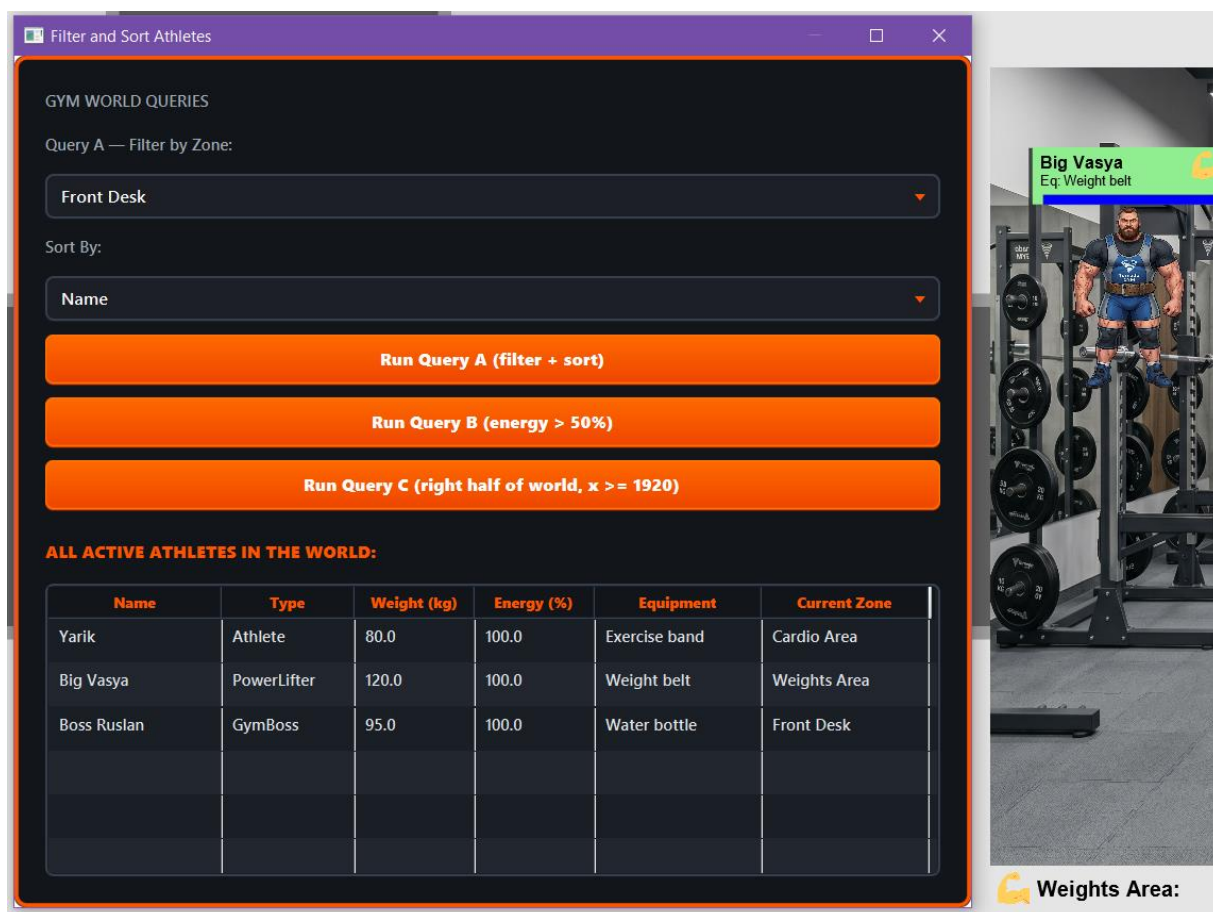
4.5 Deleting and Cloning Athletes

To delete an active athlete from the scene, press the DELETE key. This action is irreversible within the current session unless the state has been saved beforehand.

Cloning allows you to quickly create a copy of an existing athlete while preserving all of its parameters. The clone is placed next to the original, shifted to the right. To clone the active athlete, use the Ctrl+C key combination. The clone is a completely independent object and is not tied to the original's current training zone.

4.6 Sorting and Statistics

The program supports sorting the displayed list of athletes by three criteria: by name (alphabetical order), by weight, and by energy level[14, 15]. Sorting is accessed by pressing the G key on the keyboard. The sort order can be toggled between ascending and descending.



The screenshot shows a software interface for managing athletes. The main window, titled "Filter and Sort Athletes", contains a section for "GYM WORLD QUERIES". It features a dropdown menu for "Query A — Filter by Zone:" set to "Front Desk", a "Sort By:" dropdown set to "Name", and three orange buttons: "Run Query A (filter + sort)", "Run Query B (energy > 50%)", and "Run Query C (right half of world, x >= 1920)". Below these is a table titled "ALL ACTIVE ATHLETES IN THE WORLD:".

Name	Type	Weight (kg)	Energy (%)	Equipment	Current Zone
Yarik	Athlete	80.0	100.0	Exercise band	Cardio Area
Big Vasya	PowerLifter	120.0	100.0	Weight belt	Weights Area
Boss Ruslan	GymBoss	95.0	100.0	Water bottle	Front Desk

To the right of the window is a 3D model of an athlete, "Big Vasya", wearing a blue and black outfit and a weight belt, standing in a gym environment. A label above the model reads "Big Vasya Eq: Weight belt". Below the model, a yellow icon and the text "Weights Area:" are visible.

Figure 4.4 – Athlete Statistics Window

Thanks to the convenient tabular presentation (see Figure 4.4) and the ability to use special queries for filtering, the user has a powerful tool for quickly analyzing and organizing information about all active participants in the simulation.

4.7 Saving and Loading the State

The program supports saving the current state of the game field to a binary file in the .gym format (serialization of Java objects). Saving is performed by pressing the Ctrl+S key combination. A standard system dialog opens to select the save location.

Loading a previously saved state is performed using the Ctrl+L key combination. Upon loading, all current scene objects are replaced with the saved ones. It is important to note that visual components (JavaFX nodes) are not serialized—they are automatically recreated after loading via the `initVisuals()` method.

4.8 Error Messages and Special Cases

If incorrect numerical values are entered (for example, non-numeric text in the weight or energy field), the program automatically substitutes default values: weight—80 kg, energy—no change. Thus, no critical errors will occur due to incorrect input.

If the program fails to launch, the most likely causes are: missing or incompatible Java version (lower than 17), a corrupted archive file, or missing required graphics resources in the JAR file. In this case, it is recommended to run the program from the command line—this will allow you to view a detailed error message in the standard output stream.

CONCLUSIONS

As a result of this course project, the “Tornado GYM” application was developed—an interactive gym simulator with a graphical user interface built on the JavaFX platform. A three-level hierarchy of athlete micro-objects (Athlete → PowerLifter → GymBoss) and three zone macro-objects (Front Desk, Cardio Area, Weights Area) were implemented, all operating within the universal GymWorld object. During the design and implementation of the system, we successfully applied all the key principles of object-oriented programming: abstraction, encapsulation, inheritance, and polymorphism. Deep copying is implemented via the Cloneable interface, and saving the simulation state is achieved through the serialization of Java objects [8, 9]. Additionally, the use of lambda expressions and the Stream API is demonstrated for implementing queries and sorting athletes by three criteria [14, 15]. The relevance of the topic was analyzed: according to WHO data (2022)[3], more than 1.4 billion adults worldwide lead a sedentary lifestyle, which confirms the social significance of educational and motivational tools such as the developed simulator.

While working on the project, practical programming skills were acquired in the IntelliJ IDEA environment, and knowledge of the Java language and the JavaFX library for building graphical user interfaces, animations, and event handling was deepened. We gained experience in designing application architecture using UML diagrams (inheritance, aggregation, composition, association, cooperation, sequence, and state). The developed product meets all the requirements of the technical specifications: it supports a mini-map with navigation, automatic object movement, dialog boxes for controlling athletes, saving and loading the state via FileChooser, and implements at least three queries for counting micro-objects and setting sorting criteria. The experience gained serves as a valuable foundation for the further development of complex desktop applications with rich graphical interfaces on the Java platform.

LIST OF REFERENCES

1. Cózar-Gutiérrez R. et al. Positive and Negative Impacts of Gamification on the Fitness Industry. *European Journal of Investigation in Health, Psychology and Education*. 2023. Vol. 13, no. 8. P. 1342–1356. URL: <https://doi.org/10.3390/ejihpe13080103> (accessed June 1, 2026).
2. Mazeas A. et al. Evaluating the Effectiveness of Gamification on Physical Activity: Systematic Review and Meta-analysis of Randomized Controlled Trials. *Journal of Medical Internet Research*. 2022. Vol. 24, no. 1. P. e26779. URL: <https://doi.org/10.2196/26779> (accessed June 1, 2026).
3. World Health Organization. *Global Status Report on Physical Activity 2022*. Geneva: WHO, 2022. 92 p. URL: <https://www.who.int/publications/i/item/9789240059153> (accessed June 1, 2026).
4. Sharan K. *Learn JavaFX 17: Building User Experience and Interfaces with Java*. 2nd ed. New York: Apress, 2022. 1,100 p.
5. Deitel, P., and Deitel, H. **Java How to Program**. 11th ed. Hoboken: Pearson, 2018. 1,006 pp.
6. Oracle Corporation. *JavaFX Overview (Release 8)*. Oracle Documentation. 2024. URL: <https://docs.oracle.com/javase/8/javafx/get-started-tutorial/jfx-overview.htm> (accessed June 1, 2026).
7. OpenJFX. *JavaFX Documentation*. OpenJFX Community. 2024. URL: <https://openjfx.io/javadoc/17/> (accessed June 1, 2026).
8. Oracle Corporation. *Java Object Serialization*. Oracle Documentation. 2024. URL: <https://docs.oracle.com/javase/8/docs/technotes/guides/serialization/index.html> (accessed June 1, 2026).
9. Oracle Corporation. *Serializable (Java SE 17 & JDK 17)*. Oracle API Documentation. 2024. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/Serializable.html> (accessed June 1, 2026).
10. Goncalves, A. **Beginning Java EE 7**. New York: Apress, 2013. 608 p.

11. Fowler, M. **UML Distilled: A Brief Guide to the Standard Object Modeling Language**. 3rd ed. Boston: Addison-Wesley, 2003. 208 pp.
12. Bloch, J. **Effective Java**. 3rd ed. Boston: Addison-Wesley, 2018. 412 pp.
13. Gamma, E., et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston: Addison-Wesley, 1994. 395 pp.
14. Oracle Corporation. The Java Tutorials — Lambda Expressions. Oracle Documentation. 2024. URL: <https://docs.oracle.com/javase/tutorial/java/javaOO/lambdaexpressions.html> (accessed June 1, 2026).
15. Oracle Corporation. Stream (Java SE 17 & JDK 17). Oracle API Documentation. 2024. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/stream/Stream.html> (accessed June 1, 2026).